CSCI 5800 Generative AI
Spring 2025

Assignment 1: Summarizing Unstructured, Natural-Language Data
Deadline: April 18, 2025 – 11:59 PM
Total Points: 100

Large Language Models (LLMs) are extremely flexible tools and are effective in tasks that require understanding human language. Their pretraining procedure, which involves next token prediction on vast amounts of text data, enables them to develop an understanding of syntax, grammar, and linguistic meaning. One strong use case is the ability to analyze large amounts of unstructured text data, allowing us to derive insights more efficiently.

Lab Context

This assignment will focus on leveraging LLM capabilities in the context of restaurant reviews. We provide the file 'restaurant-data.txt' which contains all of the restaurant reviews required for this task. These reviews are qualitative. Within the rest of the lab, you will be using the AutoGen framework to fetch restaurant reviews, summarize these reviews to extract scores for each review, and finally aggregate these scores. Your end solution will be able to answer queries about a restaurant and give back a score.

Here are some example queries:

- "How good is Subway as a restaurant?"
- "What would you rate In N Out?"

Note that the restaurant names provided in the queries may not exactly match the exact syntax in `restaurant-data.txt` (like the query having the restaurant name in lowercase). We will only run queries on valid restaurants, so no queries that contain invalid restaurants/structures will be tested.

The format is that each review is on a new line, and each line begins with the restaurant name. You'll notice that each review is qualitative, with no mention of ratings, numbers, or quantitative data. However, each review has a series of adjectives that nicely correspond to the ratings 1, 2, 3, 4, and 5, allowing you to easily associate a score for food quality and service quality for each restaurant.

AutoGen Framework

AutoGen is a framework that enables the creation of multi-agent workflows involving multiple LLMs. Essentially, you can think of it as a way to define "control flows" or "conversation flows" between multiple LLMs. This way, you can chain together several individual LLM agents, having them all work together by conversing with each other to accomplish a larger task. Through AutoGen, users can define networks of LLM agents, enabling complex reasoning, self-evaluating, data processing pipelines, and much more. Additional Features of AutoGen.

For this assignment, we recommend getting a sense of the common conversation patterns. Being familiar with the patterns of inter-agent communication will allow you to better architect the system needed for the assignment. Here are some of the [Common Conversation Patterns]:

- Two-Agent Chats
- Sequential Chats
- Group Chat

We also suggest reading the reference for the `ConversableAgent` class. [Docs Link]. Particularly, we suggest reading the functions related to `initiate_chat`, the arguments for this, and the `result` that will be returned from calling this function.

For the entirety of the assignment, we **strongly recommend using the GPT-4o-mini model** due to its cost efficiency. It will be more than 10 times cheaper than using GPT-4o while providing "similar" levels of intelligence (at least for the purpose of this assignment), so you can expect to incur a much lower cost.

Some notable features and keywords of AutoGen that may be helpful in solving the assignment:

- Termination Message
- Max Messages Per Round
- Summary Method
- Summary Args

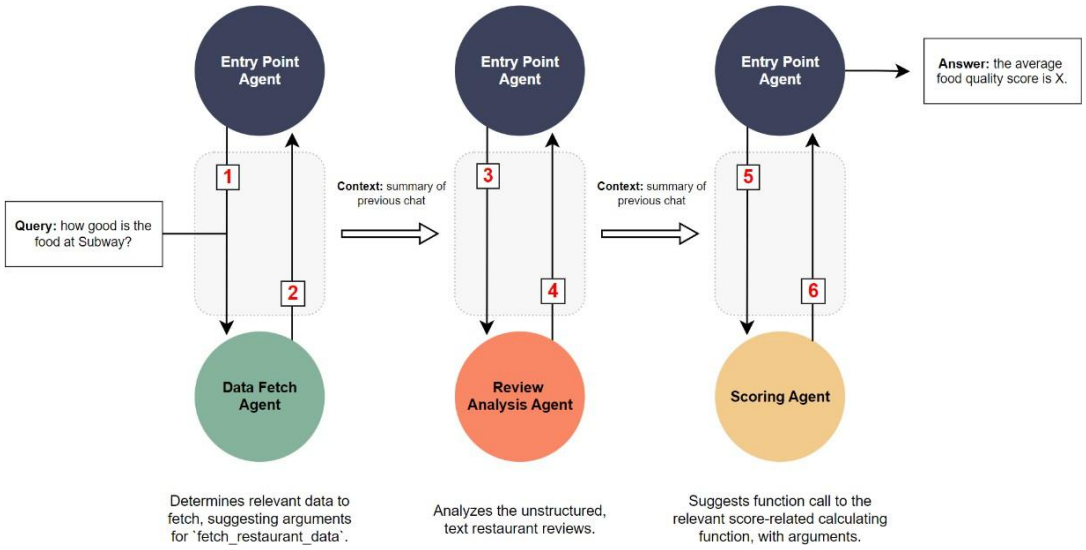
Lab Setup: Environment Variables, Virtual Environment, and Dependencies

We recommend using a virtual environment to complete this lab, then installing all the packages listed in 'requirements.txt'. Please create your own API Key and do not leak your

OpenAI API key. To create your own API Key, you can look at the following documentation: https://platform.openai.com/docs/quickstart. Using the GPT-4o-mini model, we expect the cost of this lab to be < $1 (you get a 5$ free credit when you first sign up). Store your API key as an environment variable named `OPENAI_API_KEY` and use this alias for the entirety of the assignment.

Task Description

Recommended Approach



The above figure is a diagram of the recommended architecture for this assignment. It follows a sequential conversation pattern between two agents. We choose this approach because it's the simplest solution: the pipeline is essentially a directed graph, where we first fetch the restaurant reviews, analyze them, then call a function, but with an additional "supervising" entrypoint agent.

The entry point agent is always the agent responsible for initiating the chat with other agents. Relevant summaries of previous chats between agent pairs are carried over as contexts to other chats. For example, the fetched reviews may be carried over as context from the first conversation to the second. To learn more about summaries, please refer to the `summary_method` in AutoGen.

Starter Code

- `main.py`: where the implementation primarily will go. The starter code is already annotated.

- `test.py`: public tests. Can run `python test.py`, which will print out the test result summaries. Use these as a sanity check to ensure your implementation is working as intended.
- `calculate_overall_score`: function for determining the overall score of a restaurant.
- `requirements.txt`: contains all of the necessary packages the lab requires.

Task 1: Fetching the Relevant Data

The first step is to figure out which restaurant review data we need. We should analyze the query using the data fetch agent to determine the correct function call to the fetch function. This data fetch agent will suggest a function call with particular arguments. Then, the entry point agent will execute the suggested function call. By the end of this section, you should be able to fetch the correct reviews for the appropriate restaurants.

More explicitly, you will need to write the prompts for the entrypoint agent, register the function `fetch_restaurant_data` for calling/execution, fill out the function `fetch_restaurant_data`, and determine the right arguments for initiating the chat with the entrypoint agent.

Hint: We will not test invalid queries. If a restaurant is not within the data set, it is fine. We will not test such a case.

Hint: It will be worthwhile to understand the different termination conditions in the AutoGen framework (refer to the AutoGen documentation).

Task 2: Analyzing Reviews

The next step is creating an agent that can analyze the reviews fetched in the previous section. More specifically, this agent should look at every single review corresponding to the queried restaurant and extract two scores:

- `food_score`: the quality of food at the restaurant. This will be a score from 1-5.
- `customer_service_score`: the quality of customer service at the restaurant. This will be a score from 1-5.

The agent should extract these two scores by looking for keywords in the review. Each review has keyword adjectives that correspond to the score that the restaurant should get for its `food_score` and `customer_service_score`. Here are the keywords the agent should look out for:

- Score 1/5 has one of these adjectives: awful, horrible, or disgusting.
- Score 2/5 has one of these adjectives: bad, unpleasant, or offensive.
- Score 3/5 has one of these adjectives: average, uninspiring, or forgettable.
- Score 4/5 has one of these adjectives: good, enjoyable, or satisfying.
- Score 5/5 has one of these adjectives: awesome, incredible, or amazing.

Each review will have exactly only two of these keywords (adjective describing food and adjective describing customer service), and the score (N/5) is only determined through the above-listed keywords. No other factors go into score extraction. To illustrate the concept of scoring better, here's an example review.

> *The food at McDonald's was average, but the customer service was unpleasant. The uninspiring menu options were served quickly, but the staff seemed disinterested and unhelpful.*

We see that the food is described as "average", which corresponds to a `food_score` of 3. We also notice that the customer service is described as "unpleasant", which corresponds to a `customer_service_score` of 2. Therefore, the agent should be able to determine `food_score: 3` and `customer_service_score: 2` for this example review.

We provide the data of all restaurant reviews in the file `restaurant-reviews.txt`. It has the following format:

<restaurant_name>. <review>.

<restaurant_name>. <review>.

<restaurant_name>. <review>.

...

<restaurant_name>. <review>.

By the end of this section, your agent should have extracted two scores (`food_score` and `customer_service_score`) for **every** review associated with the restaurant in the query. All reviews for every restaurant should be able to fit within the context window, so you can summarize them all at once.

To accomplish this, you need to write the prompt so that this agent can score each review, and you need to initiate the chat with the review analyzer agent (add an argument dictionary to `entrypoint_agent.initiate_chats`).

Hint: For this agent, it will help to explicitly list down keywords corresponding to food and customer service. Prompt your agent so that it enumerates through the reviews, extracts the keyword associated with food, extracts the keyword associated with customer service, and then creates two scores (food and customer service) for that review.

Hint: It will be worthwhile to look into different arguments for `summary_method` (refer to the AutoGen documentation)

The final step involves creating an agent that aggregates the extracted scores and calls the function 'calculate_overall_score' to determine the final rating for each restaurant.

Task 3: Scoring Agent

The final step is creating a scoring agent. This agent will need to look at all of the review's `food_score` and `customer_service_score` to make a final function call to `calculate_overall_score`. This agent should be provided a summary with enough context from the previous chat to determine the arguments for the function `calculate_overall_score`. Complete the implementation for the scoring function `calculate_overall_score` with the descriptions provided in the comments. Please don't modify the function signature or the return format. Also, write the necessary prompts for this agent. At the end of this section, you should be fully complete. You should be able to answer queries regarding any restaurant in the list of restaurants.

Hint: It will be worthwhile to look into different arguments for `summary_method`. It will also be worthwhile to re-read the documentation on function calling.

Testing, Submission, and Grading

We provide a set of public tests within the script `test.py`, which can be used as a sanity check for the lab implementation. If you successfully pass all tests, it means your implementation had the right score for the provided test cases and is fully functioning.

Zip your entire project folder, and upload the folder into Canvas.